

Оптимизируем операторы K8s

А зачем?

Чуриков Анатолий

Инженер архитектурных решений



СФЛАНТ

Синергия опыта вендора, интегратора, сервисной и консалтинговой компании



Deckhouse — продуктивное подразделение, разработчик продуктов для построения надёжной enterprise-инфраструктуры



DaaS — комплексное DevOps-сопровождение инфраструктуры в режиме 24/7 силами выделенной DevOps-команды



«Экспресс 42» — DevOps-консалтинг. От анализа узких мест в ИТ-процессах до создания роадмапа изменения ИТ для реализации цифровой трансформации

План доклада

- 01 Что такое операторы и контроллеры
- 02 Какие есть инструменты для разработки
- 03 Почему дефолтные настройки информеров нужно менять
- 04 Как избавиться от лишнего «веса» в кеше
- 05 Как настроить пробу
- 06 Айсберг

CodeFest16

Кто писал
операторы ?



Что такое операторы и контроллеры?

Чем занимается controller?

- 01 Получает задание (YAML, JSON)

Чем занимается controller?

- 01 Получает задание (YAML, JSON)
- 02 Сравнивает с текущим состоянием кластера

Чем занимается controller?

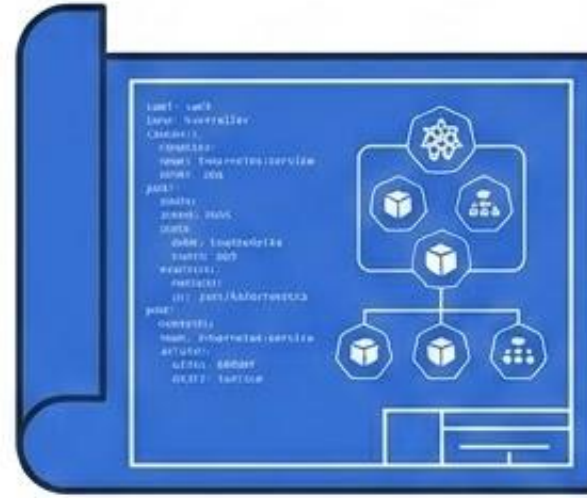
- 01 Получает задание (YAML, JSON)
- 02 Сравнивает с текущим состоянием кластера
- 03 Управляет pod, sts, service

Чем занимается controller?

- 01 Получает задание (YAML, JSON)
- 02 Сравнивает с текущим состоянием кластера
- 03 Управляет pod, sts, service
- 04 Приводит кластер в желаемое состояние

Reconciliation loop

Desired State



Controller



Actual State



Что делает оператор?

01 Управляет приложением (Kafka)

Что делает оператор?

- 01 Управляет приложением (Kafka)
- 02 Расширяет API (CRD)

Что делает оператор?

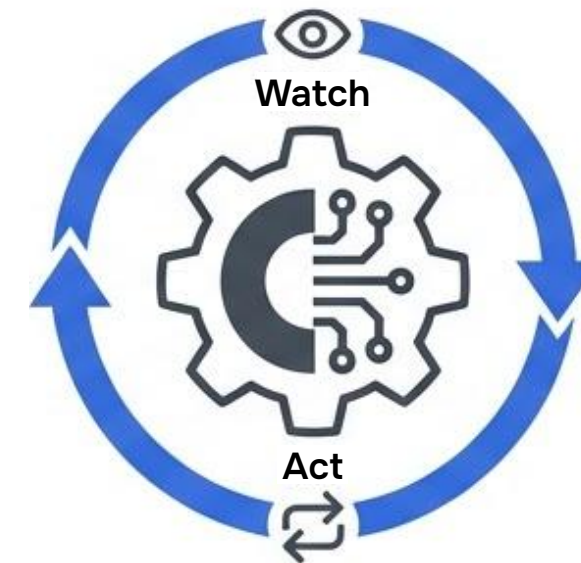
- 01 Управляет приложением (Kafka)
- 02 Расширяет API (CRD)
- 03 Автоматизирует сложные задачи (update, backups, rollback)

Что делает оператор?

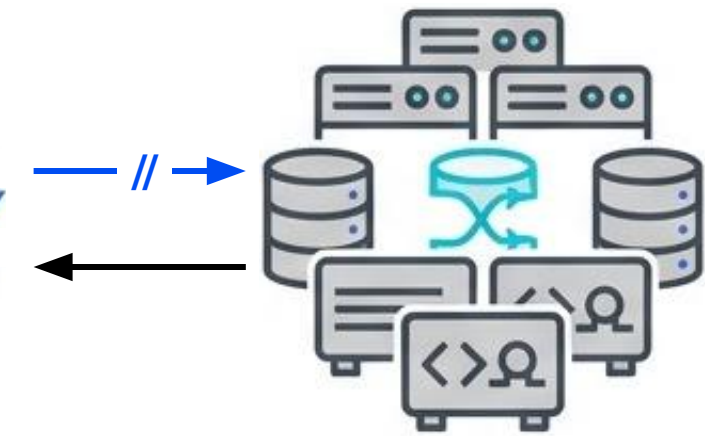
Custom
Resource
(CR)



Operator
Controller



Managed
Stateful
Application



Инструменты для разработки

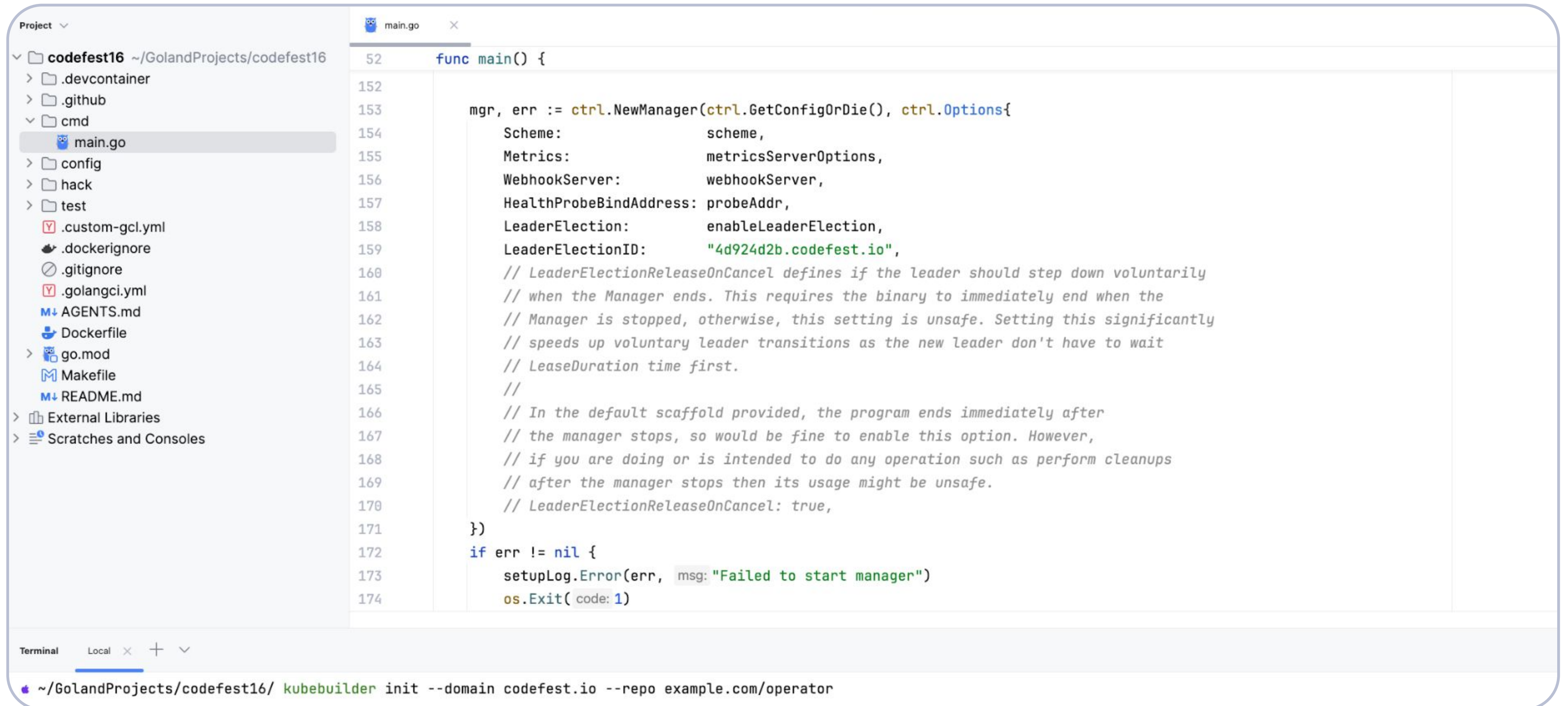
Kubebuilder

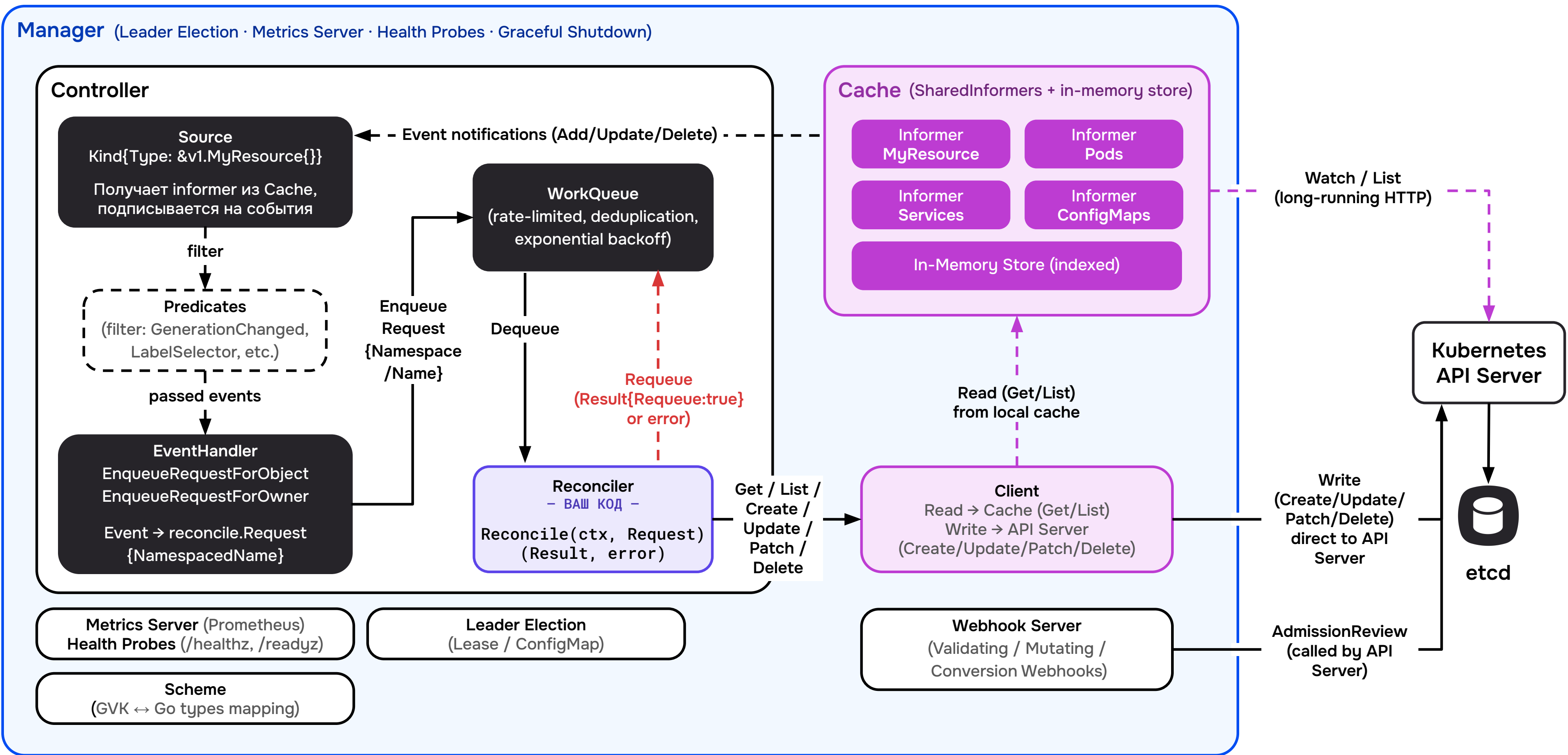
- 01 Фреймворк для создания операторов (Flux, Crossplane, AWS ACK)

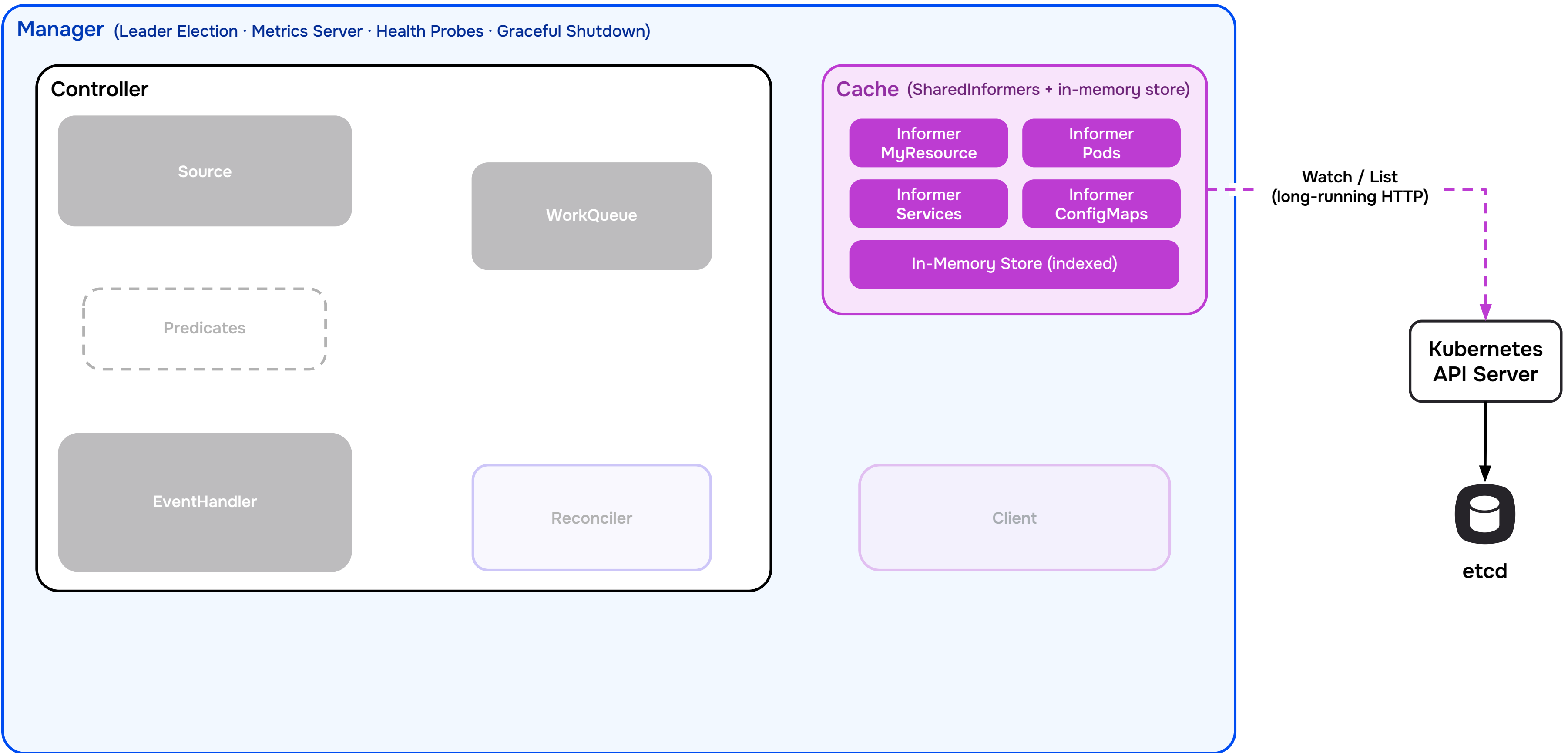
Kubebuilder

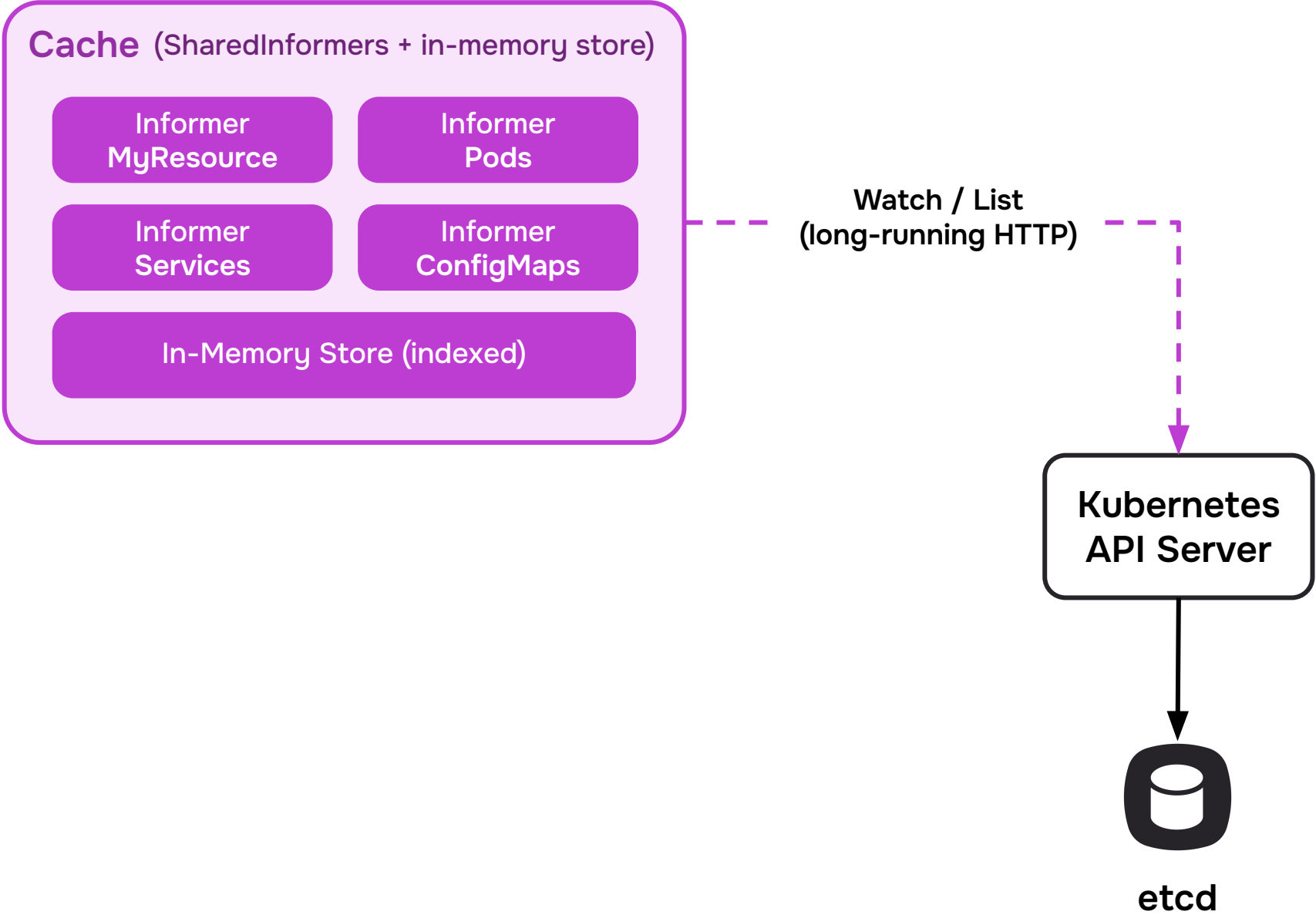
- 01 Фреймворк для создания операторов (Flux, Crossplane, AWS ACK)
- 02 Генерирует структуру проекта (файловую, CRD, вебхуки, маркеры для кодогенераторов)

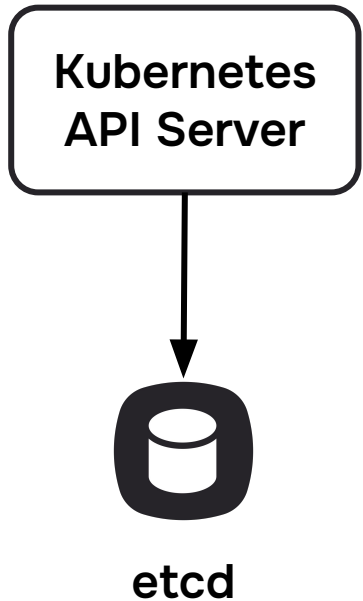
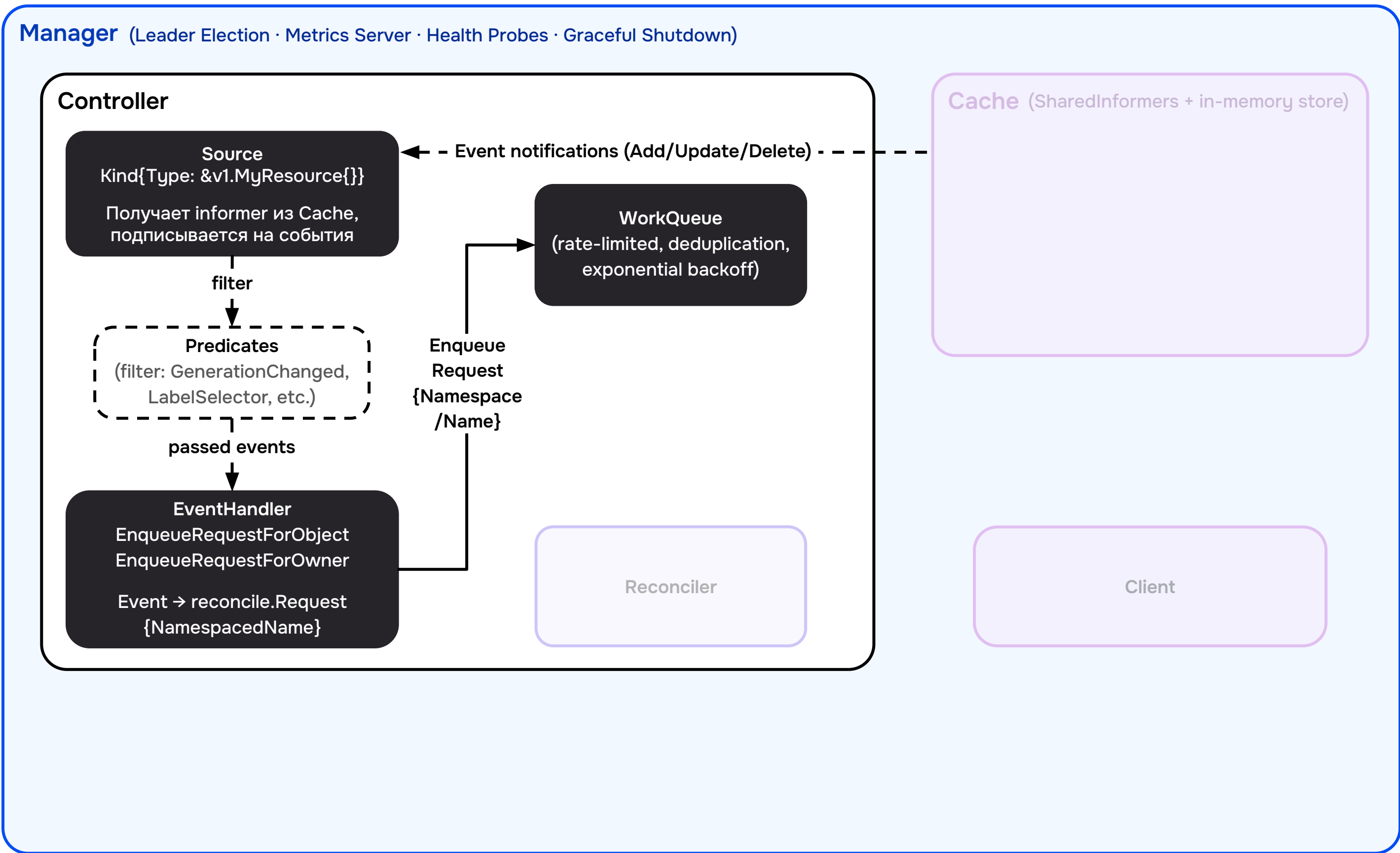
Как создать проект

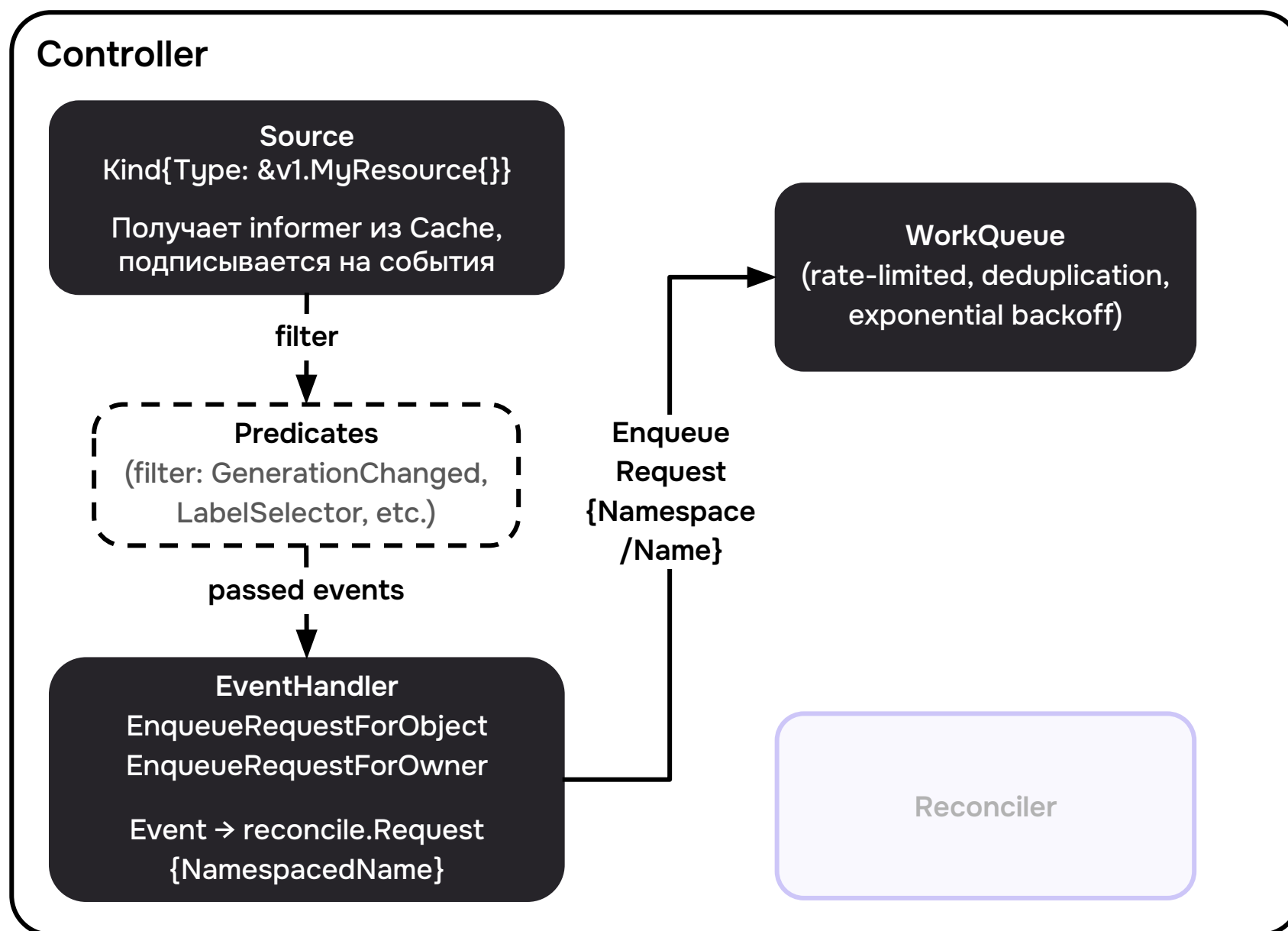












Manager (Leader Election · Metrics Server · Health Probes · Graceful Shutdown)

Controller

Source

Predicates

EventHandler

WorkQueue
(rate-limited, deduplication,
exponential backoff)

Reconciler
- ВАШ КОД -
Reconcile(ctx, Request)
(Result, error)

Get / List /
Create /
Update /
Patch /
Delete

Requeue
(Result{Requeue:true}
or error)

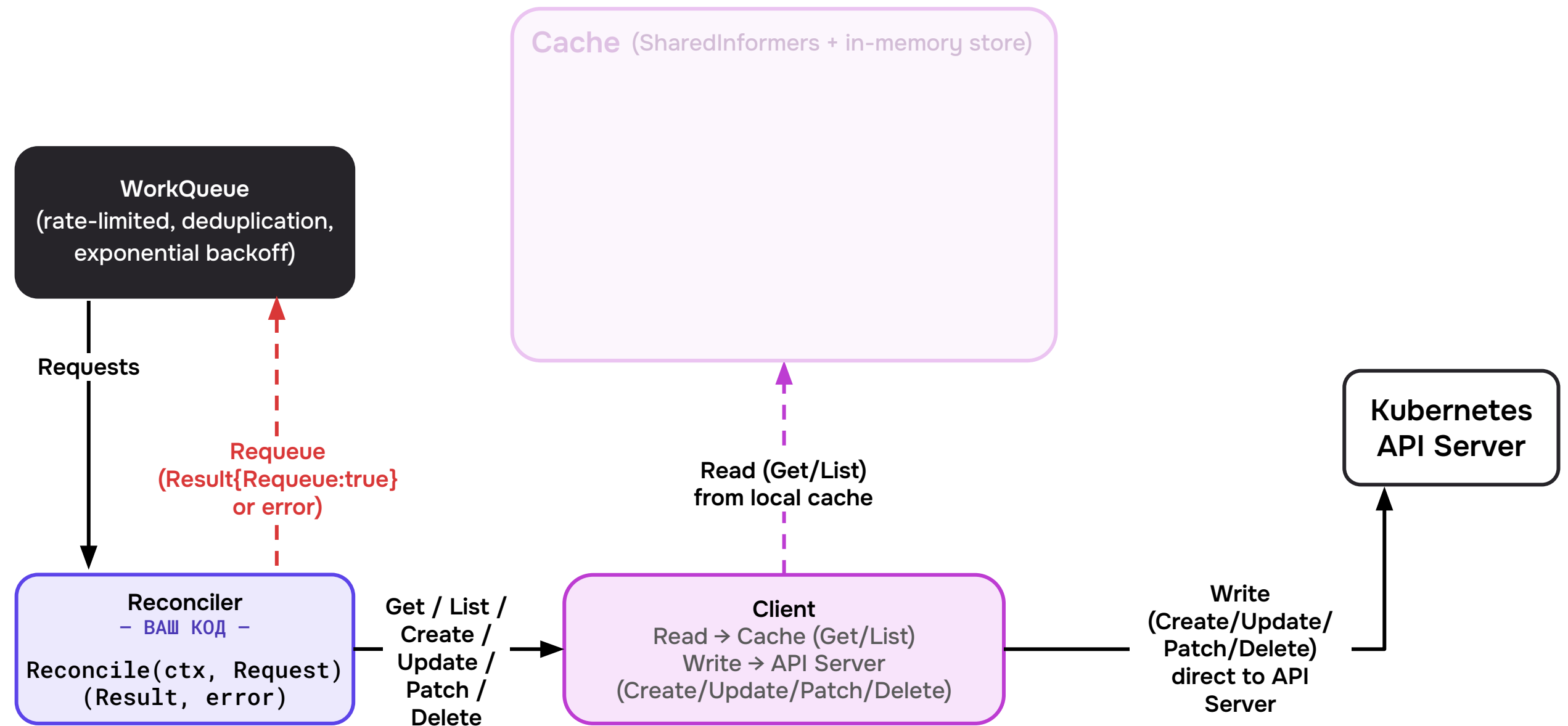
Cache (SharedInformers + in-memory store)

Read (Get/List)
from local cache

Client
Read → Cache (Get/List)
Write → API Server
(Create/Update/Patch/Delete)

Write
(Create/Update/
Patch/Delete)
direct to API
Server

**Kubernetes
API Server**





controller-runtime

kubebuilder

Вы что, и Kafka за меня сделаете?



Хьюстон, у нас проблема!



Нестабильное поведение при создании Kafka

01

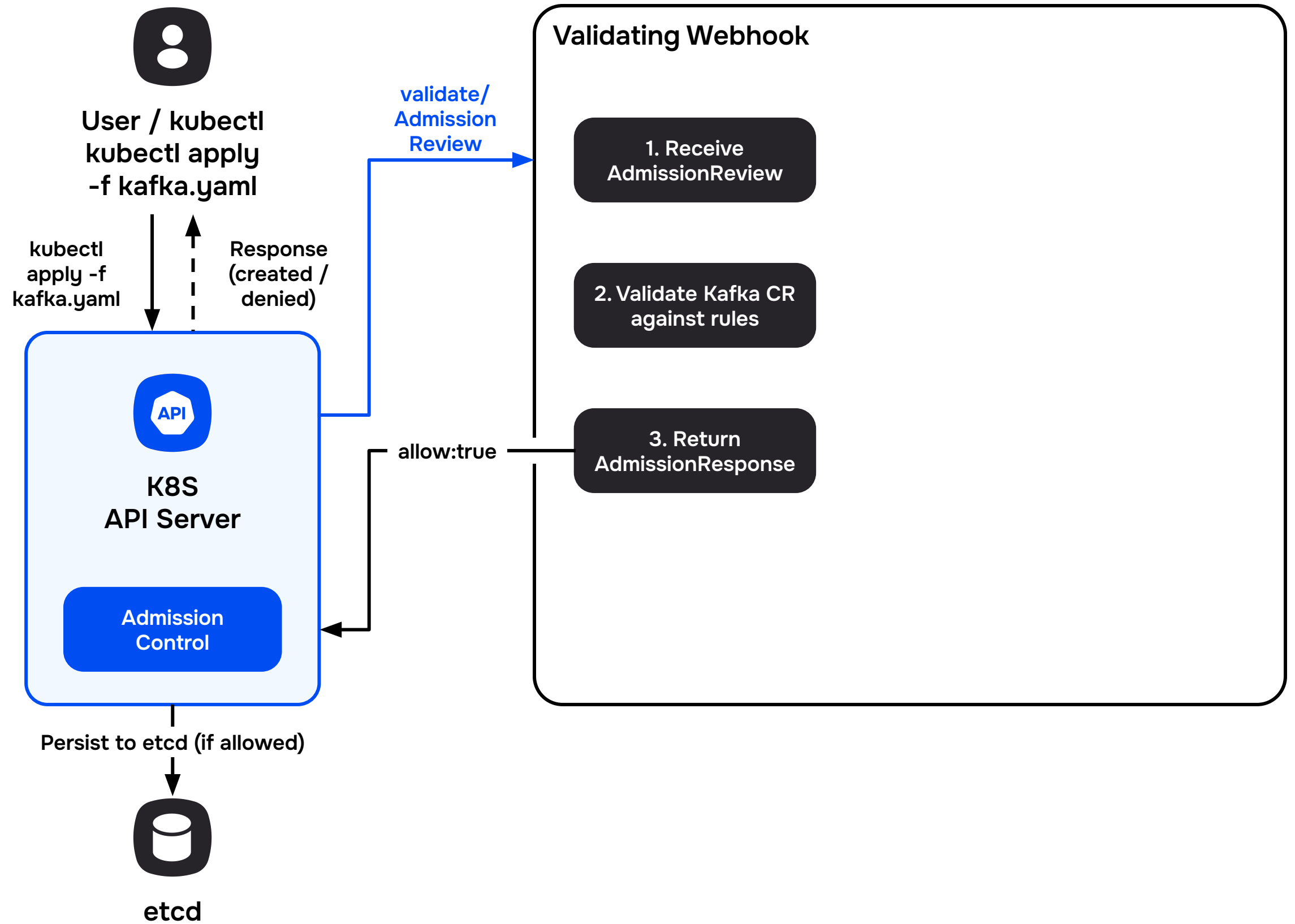
Длительное ожидание
после `kubectl apply`

Нестабильное поведение при создании Kafka

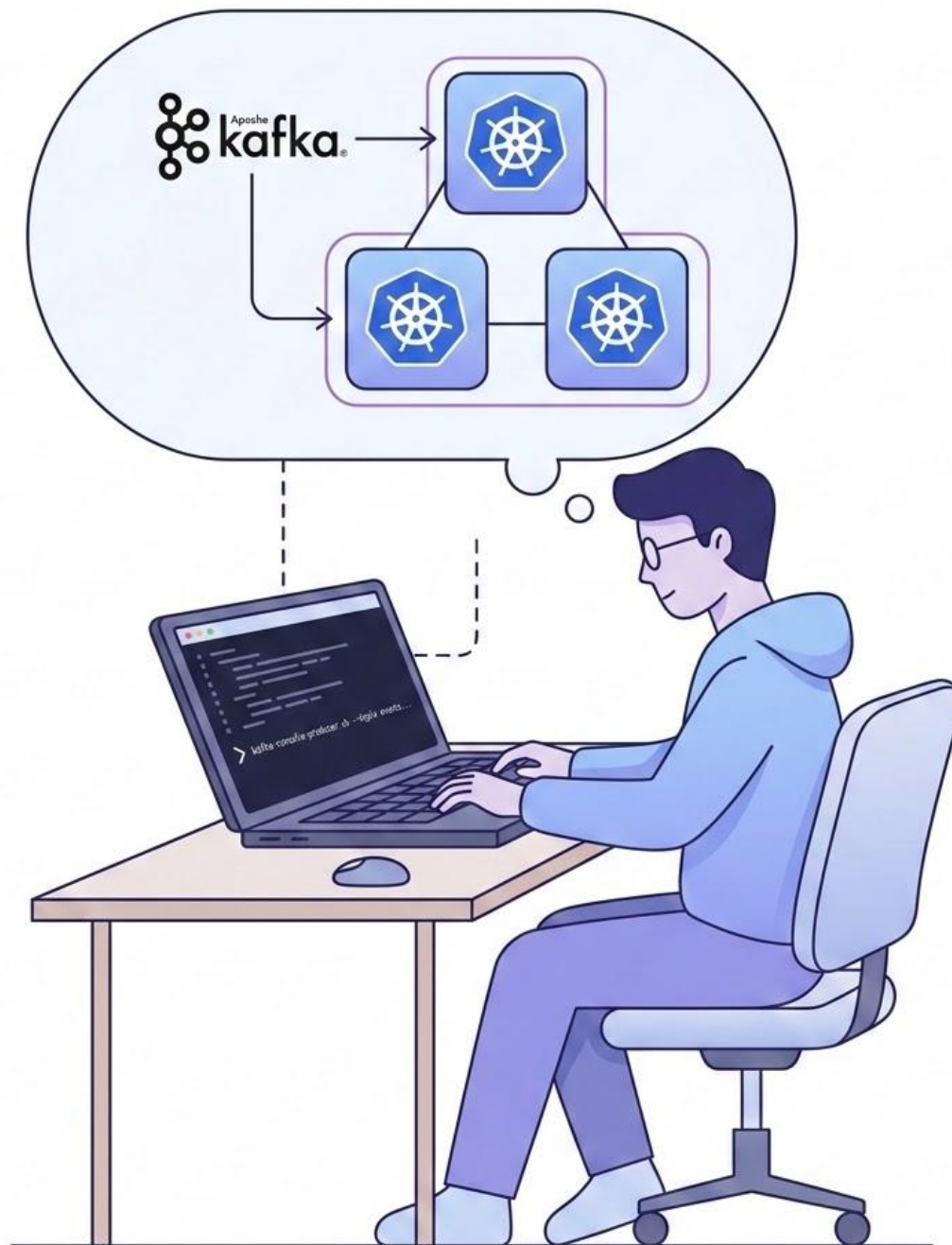
- 01 Длительное ожидание после `kubectl apply`
- 02 Возврат ошибки вместо создания кластера

Почему дефолтные настройки информеров нужно менять?

Как работает вебхук?

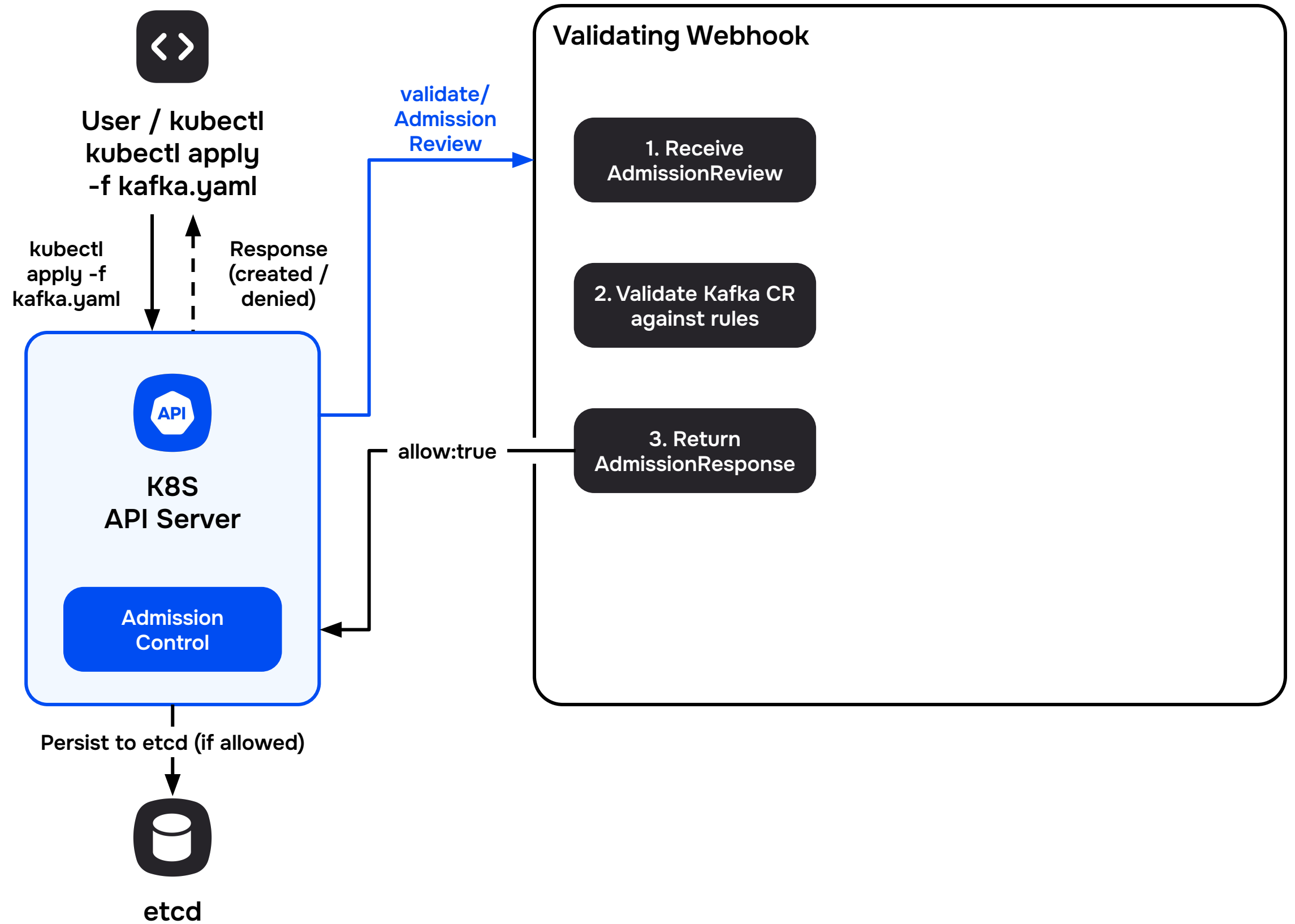


Запрос от пользователя

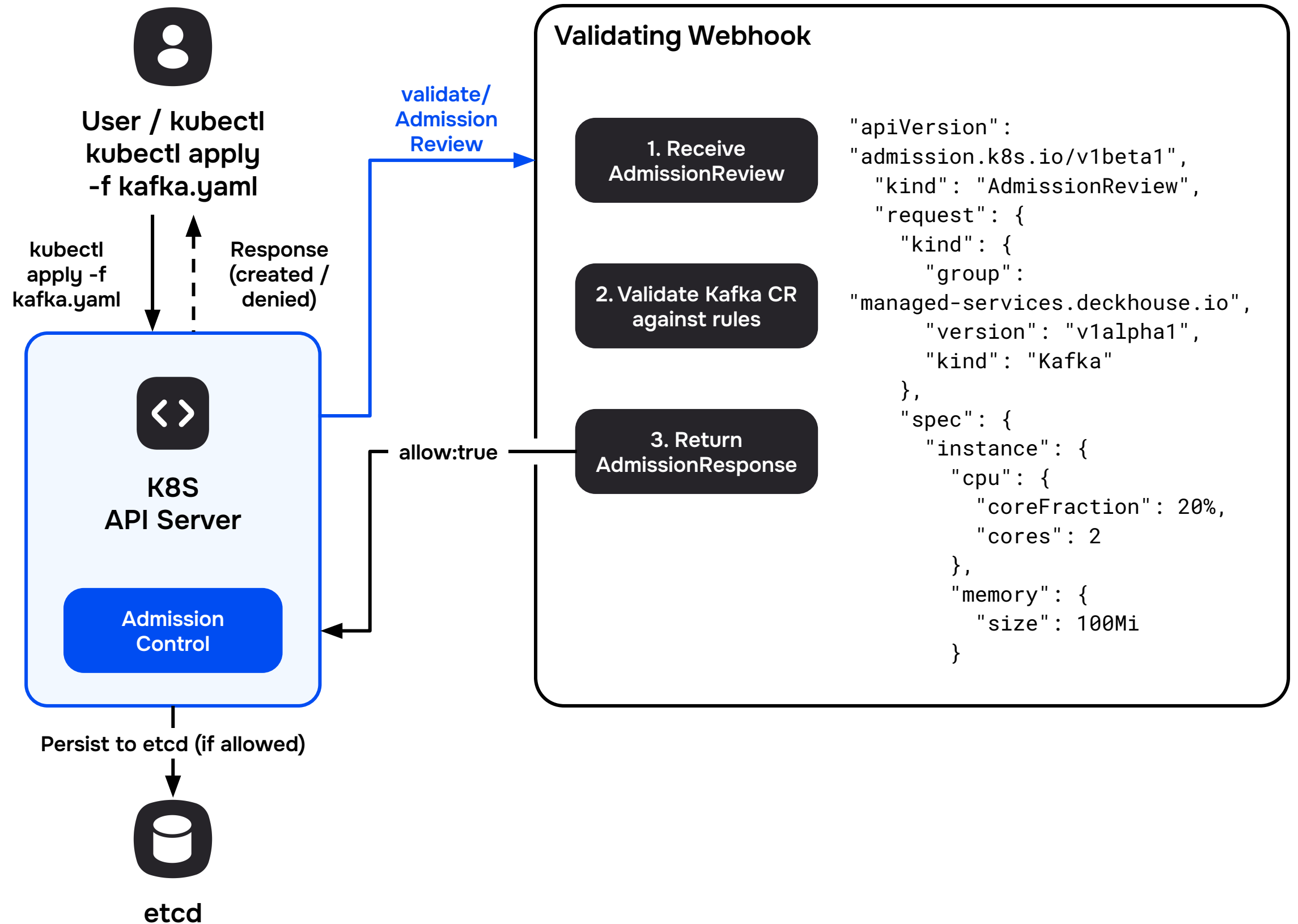


```
<> Kafka "test"
  apiVersion:managed-
  services
    .deckhouse.io/v1alpha1
kind: Kafka
spec:
  kafkaClassName:
selector
instance:
  cpu:
    cores: 1
    coreFraction: 25%
  memory: {size: 1Gi}
  pvc: {size: 500Mi}
```

Как обрабатывается запрос?



Как обрабатывается запрос?



Валидация объектов

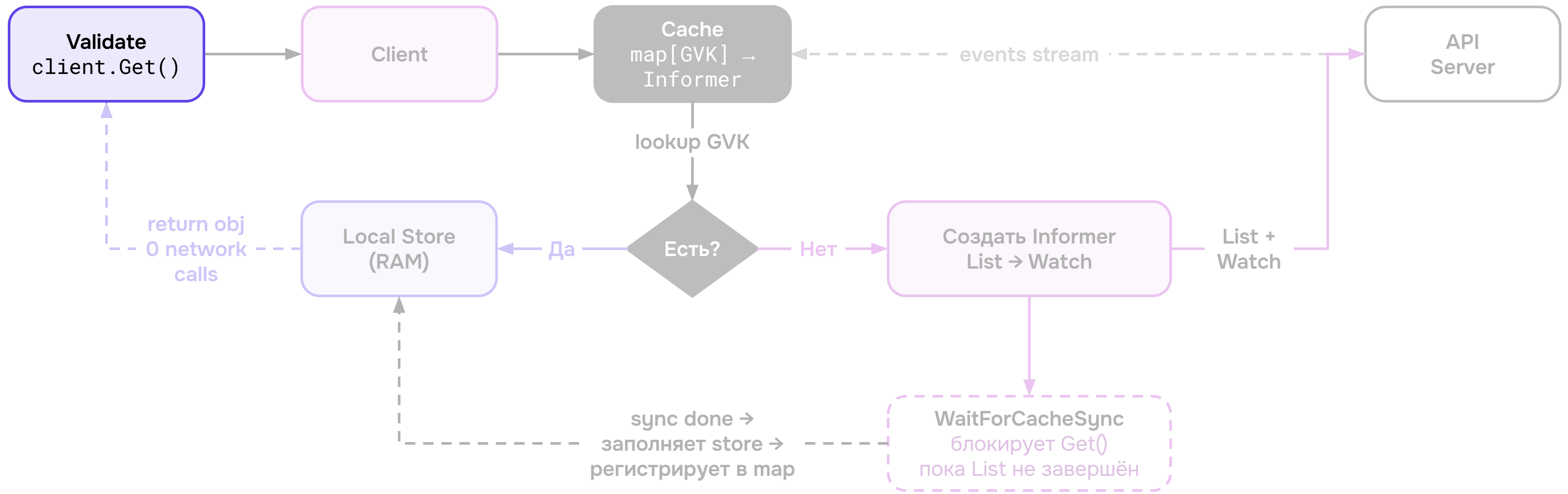


```
KafkaClass "selector"  
apiVersion: managed-services  
  .deckhouse.io/v1alpha1  
kind: KafkaClass  
spec:  
  nodeSelector:  
    hostname: stage-d8ms-  
worker-0
```

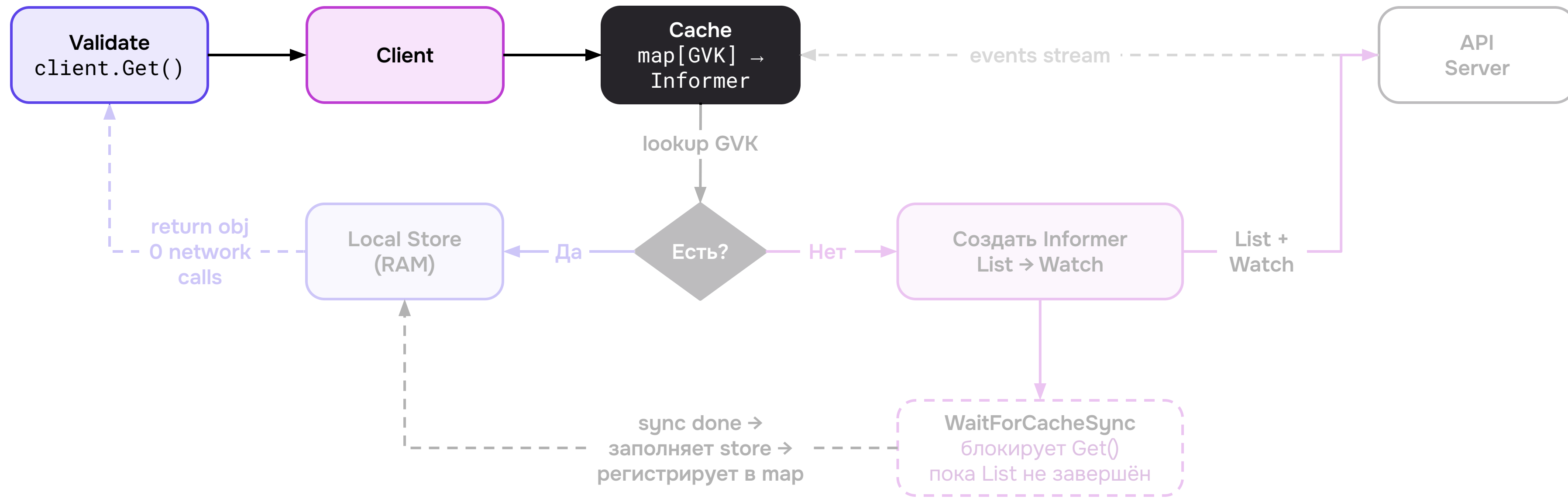
```
    sizingPolicies:  
      - cores:[1..3]  
mem:[0.5..8Gi]  
      - cores:[4..16]  
mem:[8..64Gi]  
    fractions:[50,100]%
```

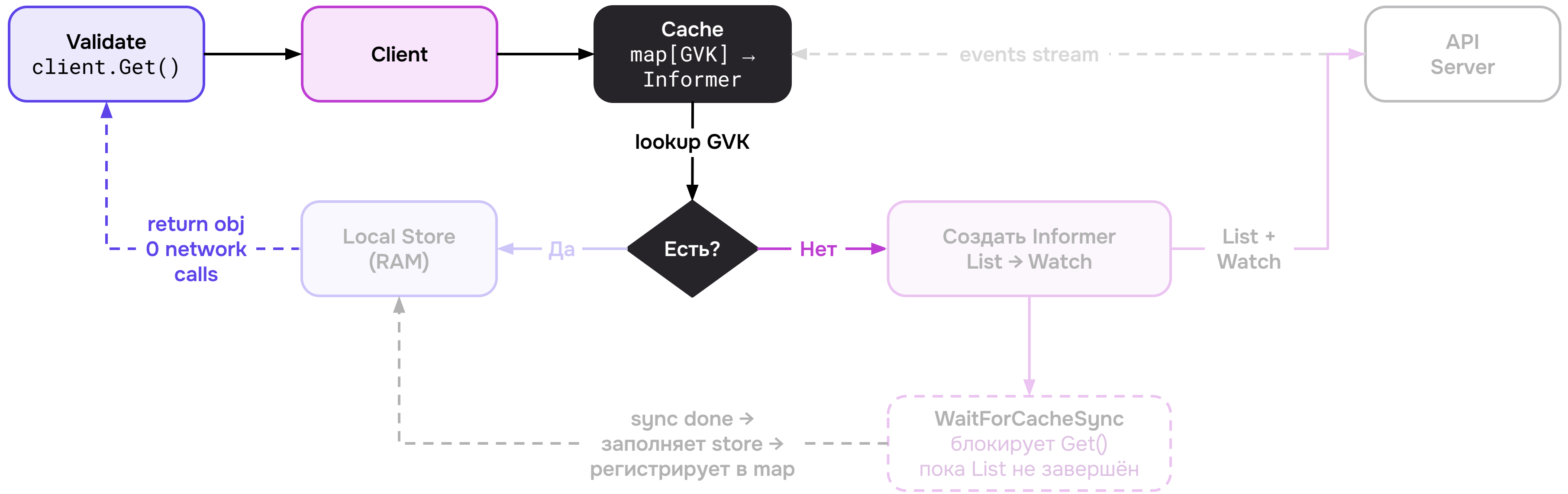
Для валидации в пользовательского Kafka
в вебхуке нужно получить kafkaclass, который
создал администратор

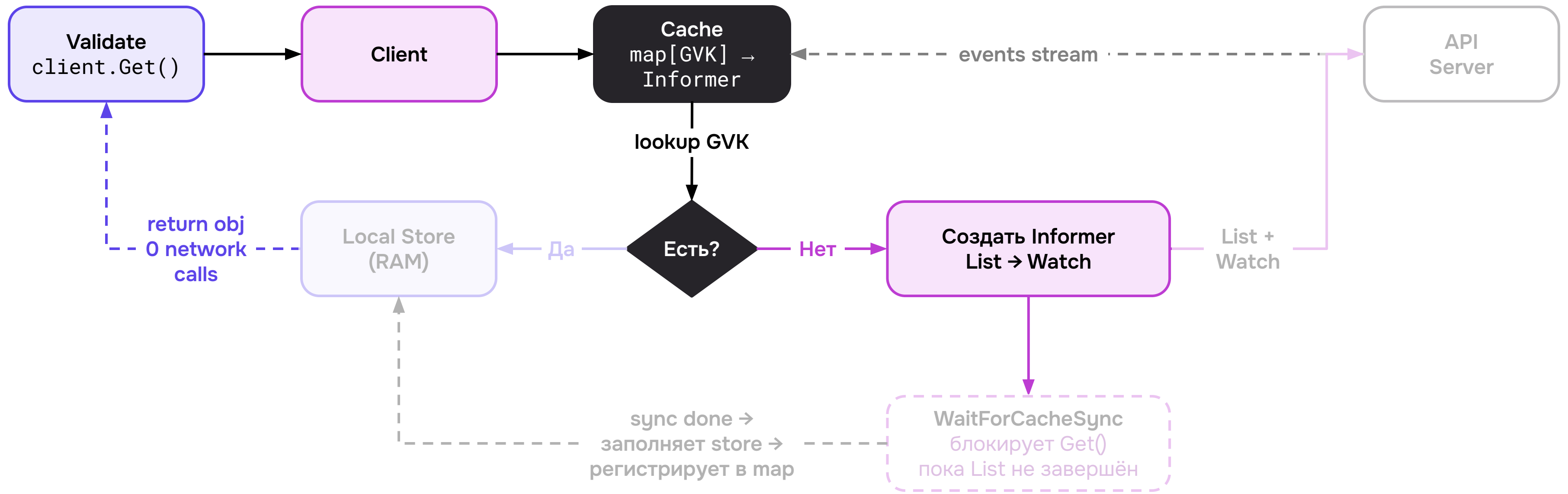
Что происходит в вебхуке?

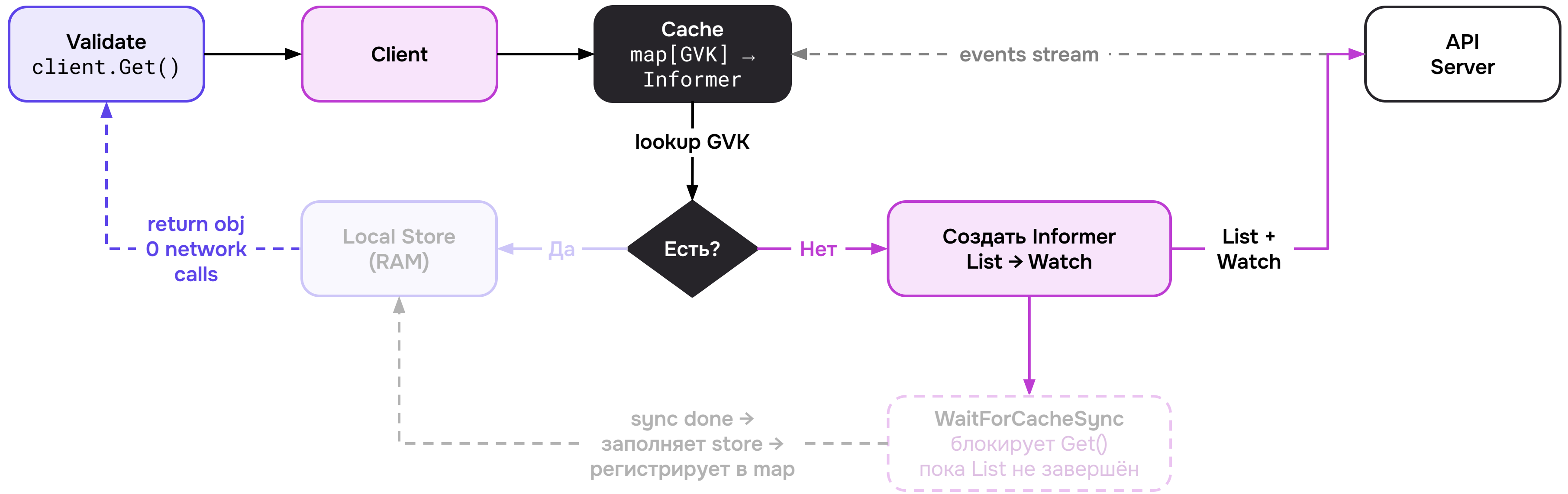


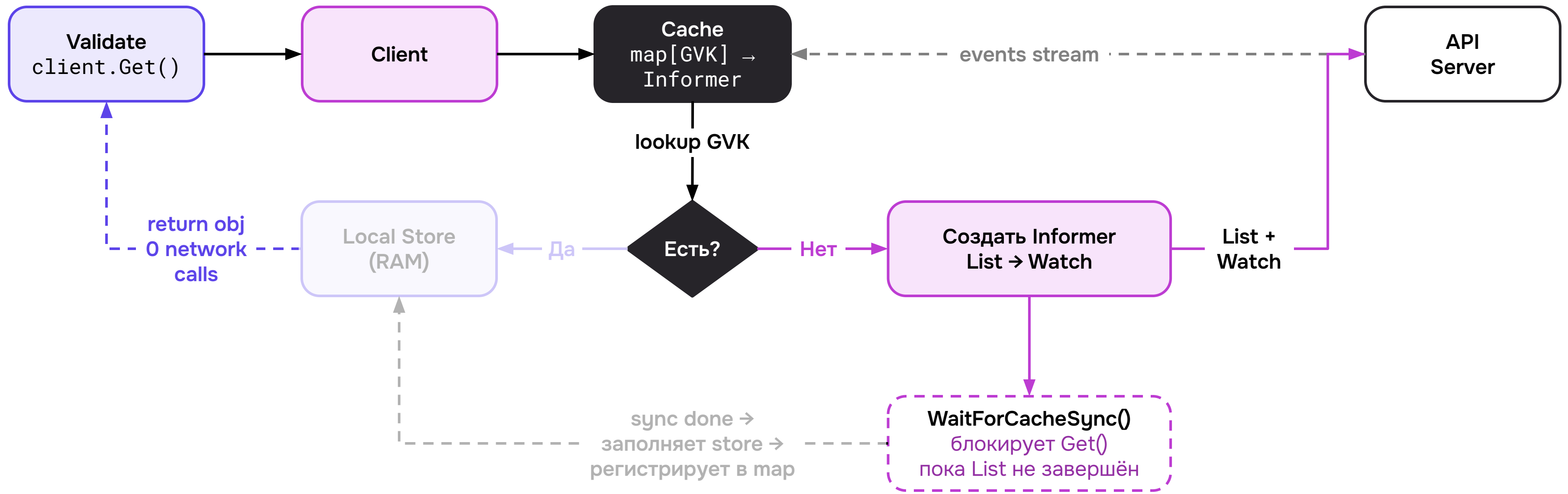


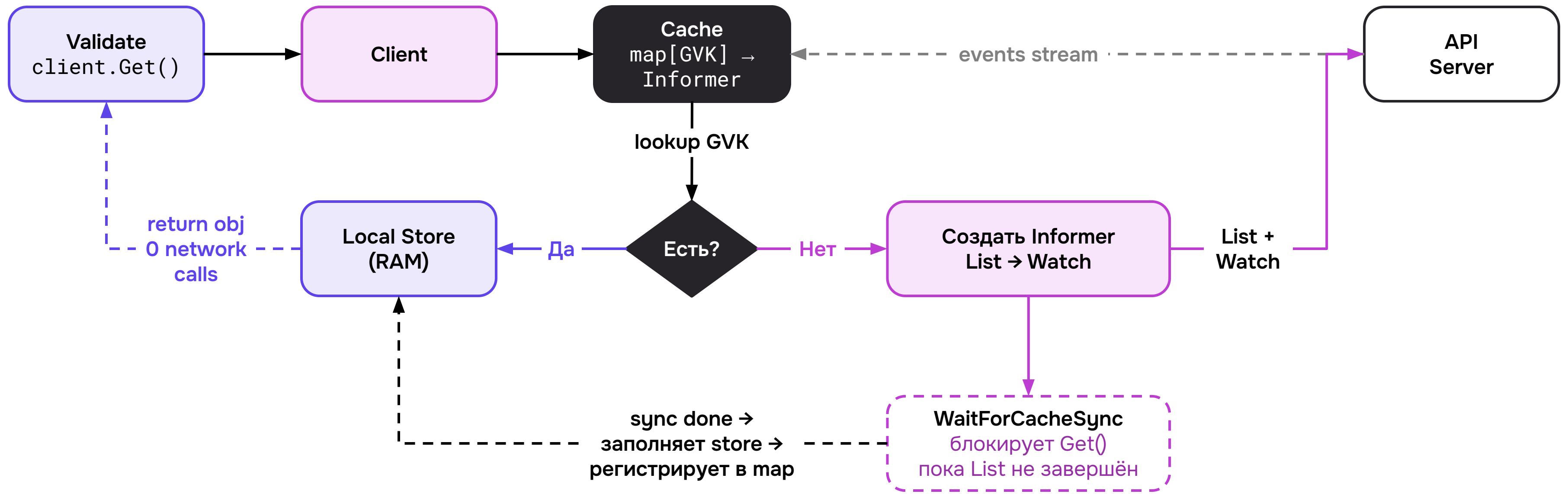












Решение – отключение динамического
создания информеров

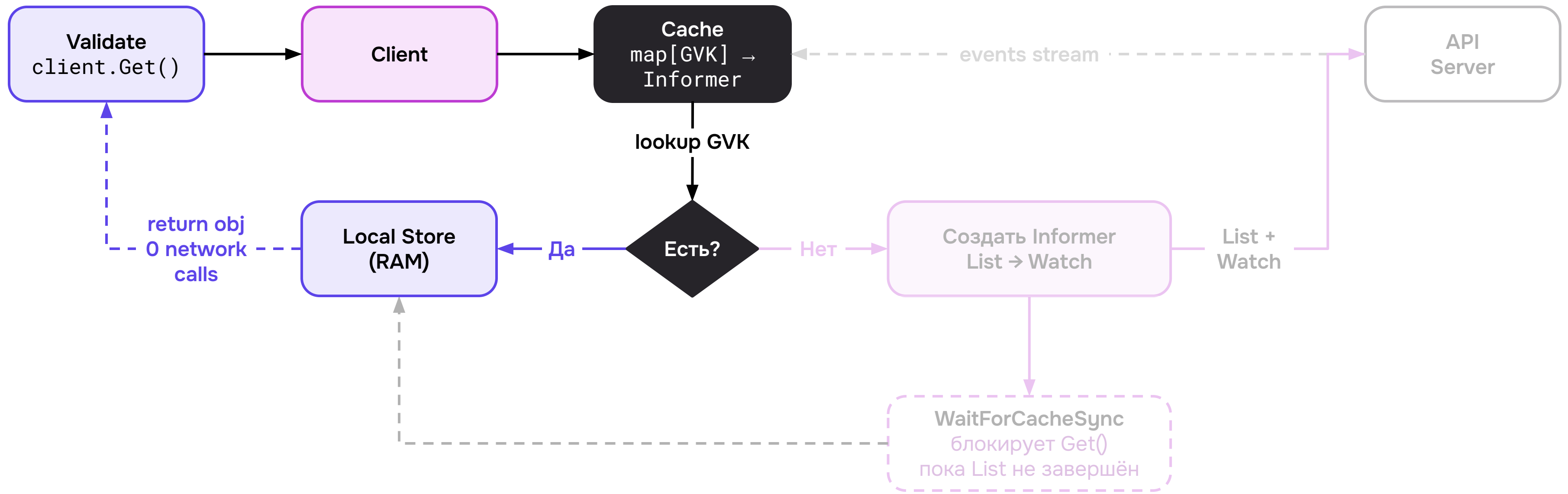
Включаем ReaderFailOnMissingInformer

```
ctrl.NewManager(cfg.Config, ctrl.Options{  
    Scheme: cfg.Scheme,  
    Cache: cache.Options{  
        ReaderFailOnMissingInformer: true },  
    }  
)
```

Что нужно сделать до запуска manager?

```
func main() {  
    mgr, err := managerbuilder.NewManager(  
        ...  
        managerbuilder.WithReaderFailOnMissingInformer(),  
    )  
  
    informerTypes := managerbuilder.NewInformerTypes(  
        &managedservicesv1alpha1.KafkaClass{},  
    )  
  
    if err := mgr.Start(ctx); err != nil {  
        log.Fatal(err)  
    }  
}
```

Как работает вебхук, если кеш уже прогрет?



Хьюстон, у нас проблема!



Что не так?

01

Время запуска зависит
от размеров кластера

Что не так?

- 01 Время запуска зависит от размеров кластера
- 02 Перезапуски подов оператора при старте

Как избавиться от лишнего веса в кеше?

Что происходит при запуске manager?

01

Менеджер обращается к информерам, зарегистрированным вручную, с использованием метода:

```
GetInformer(ctx context.Context, obj  
client.Object, opts ...InformerGetOption)  
(Informer, error).
```

Что происходит при запуске manager?

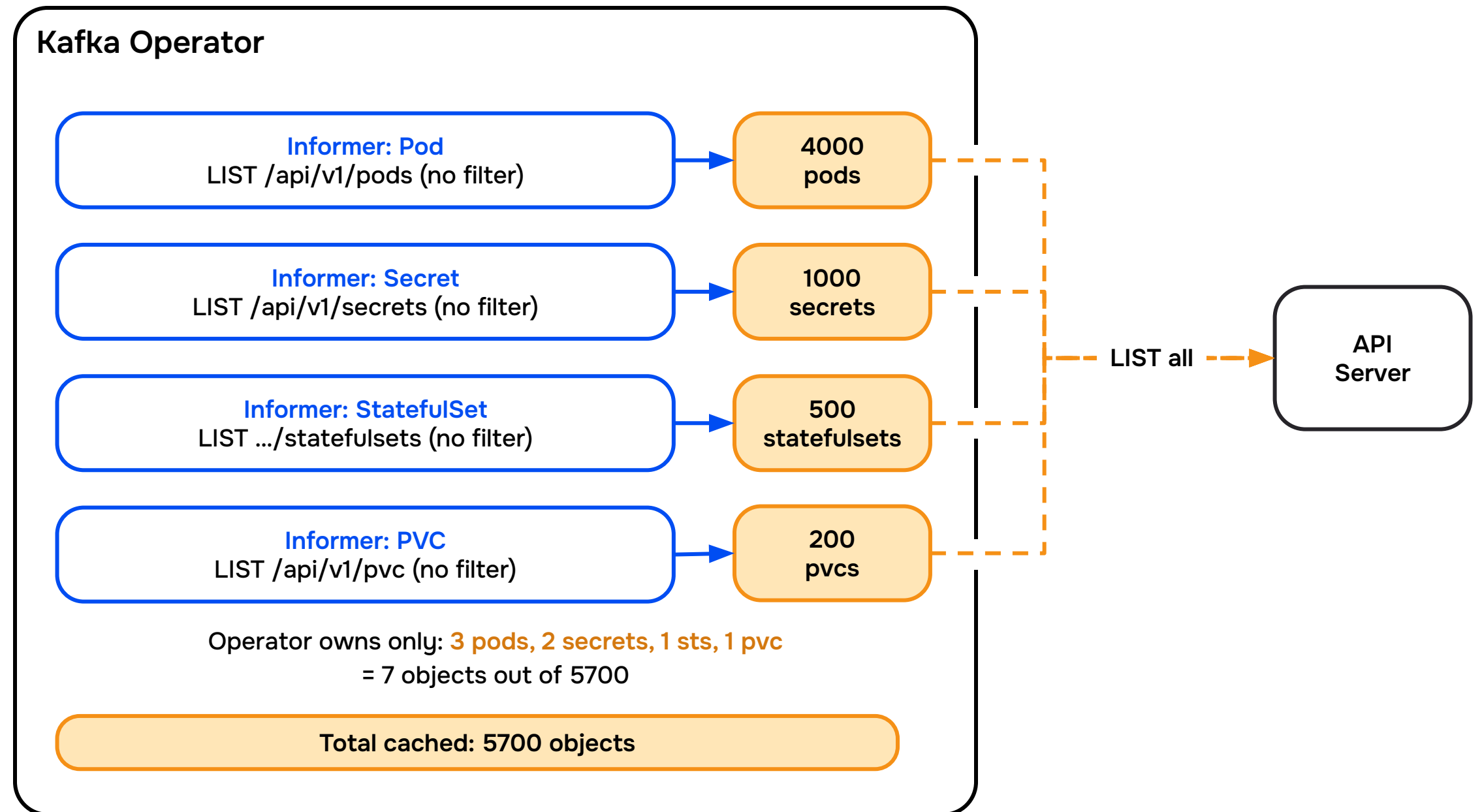
- 01 Менеджер обращается к информерам, зарегистрированным вручную, с использованием метода:

```
GetInformer(ctx context.Context, obj  
client.Object, opts ...InformerGetOption)  
(Informer, error).
```

- 02 Запускает информеры для типов, которые были зарегистрированы через:

```
ctrl.NewControllerManagedBy(mgr).  
For(&myv1.Kafka{}).  
Owns(&corev1.Pod{}).  
Complete(r)
```

При дефолтных настройках cache

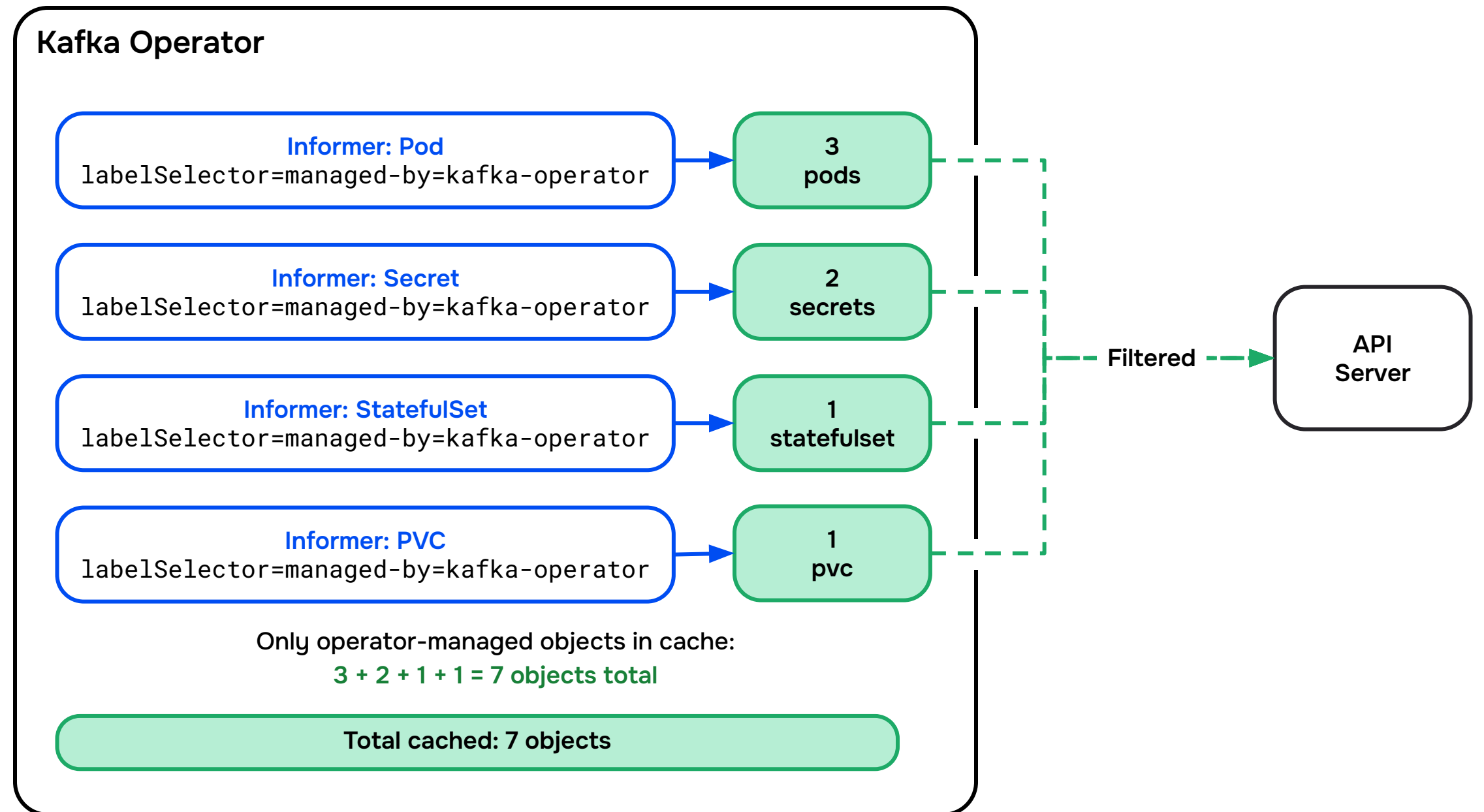


Решение — фильтрация по лейблу

```
managedBySelector := labels.SelectorFromSet(labels.Set{"managed-by": "kafka-operator"})
```

```
mgr, _ := ctrl.NewManager(cfg, ctrl.Options{  
    Cache: cache.Options{  
        ByObject: map[client.Object]cache.ByObject{  
            &corev1.Pod{}: {Label: managedBySelector},  
            &corev1.Secret{}: {Label: managedBySelector},  
            &appsv1.StatefulSet{}: {Label: managedBySelector},  
            &corev1.PersistentVolumeClaim{}: {Label: managedBySelector},  
        },  
    },  
})
```

При фильтрации по лейблу



Как проверить реальную экономию?

Проводим тестирование с использованием фильтрации по лейблу и без и замеряем использование памяти при старте

01

Используем `envtest` из `controller-runtime`

Как проверить реальную экономию?

Проводим тестирование с использованием фильтрации по лейблу и без и замеряем использование памяти при старте

01 Используем **envtest** из **controller-runtime**

02 Создаём тестовые объекты:

- 50 неймспейсов
- 100 подов в каждом
- 20 Secrets
- 4 PVC
- 2 SVC
- 1 STS

Как проверить реальную экономию?

Проводим тестирование с использованием фильтрации по лейблу и без и замеряем использование памяти при старте.

- 01 Используем `envtest` из `controller-runtime`
- 02 Создаём тестовые объекты:
 - 50 неймспейсов
 - 100 подов в каждом
 - 20 Secrets
 - 4 PVC
 - 2 SVC
 - 1 STS
- 03 «Утяжеляем» объекты (InitContainers, Sidecars, Affinity, etc.)

Как проходит тестирование?

01

Запускаем `manager`

Как проходит тестирование?

- 01 Запускаем `manager`
- 02 Вызываем сборщик `runtime.GC()`

Как проходит тестирование?

- 01 Запускаем `manager`
- 02 Вызываем сборщик `runtime.GC()`
- 03 Ждём завершения синхронизации `WaitForCacheSync()`

Как проходит тестирование?

- 01 Запускаем `manager`
- 02 Вызываем сборщик `runtime.GC()`
- 03 Ждём завершения синхронизации `WaitForCacheSync()`
- 04 Каждые 50 мс фиксируем метрики

Как проходит тестирование?

- 01 Запускаем **manager**
- 02 Вызываем сборщик **runtime.GC()**
- 03 Ждём завершения синхронизации **WaitForCacheSync()**
- 04 Каждые 50 мс фиксируем метрики
- 05 После завершения синхронизации вызываем **runtime.GC()**, считаем Heap Delta

Результаты тестирования



Всего объектов в кэше

Кэш с фильтром (текущий)

15 obj

Кэш без фильтра (сравнение)

6366 obj

Время инициализации кэша

Кэш с фильтром (текущий)

101 ms

Кэш без фильтра (сравнение)

402 ms

Дельта heap после синхронизации

Кэш с фильтром (текущий)

311.0 KiB

Кэш без фильтра (сравнение)

242.5 MiB

Чтобы избежать утечек памяти
при инициализации и, как следствие, перезапуска
подов, необходимо управлять кешем

Как взаимодействовать с K8s через пробы?

Запросы от пользователя
на создание ресурсов поступают
раньше, чем кеш был готов

Какие есть пробы и для чего?

01

Startup – даёт приложению время на долгий старт: пока probe не прошла, liveness и readiness не запускаются

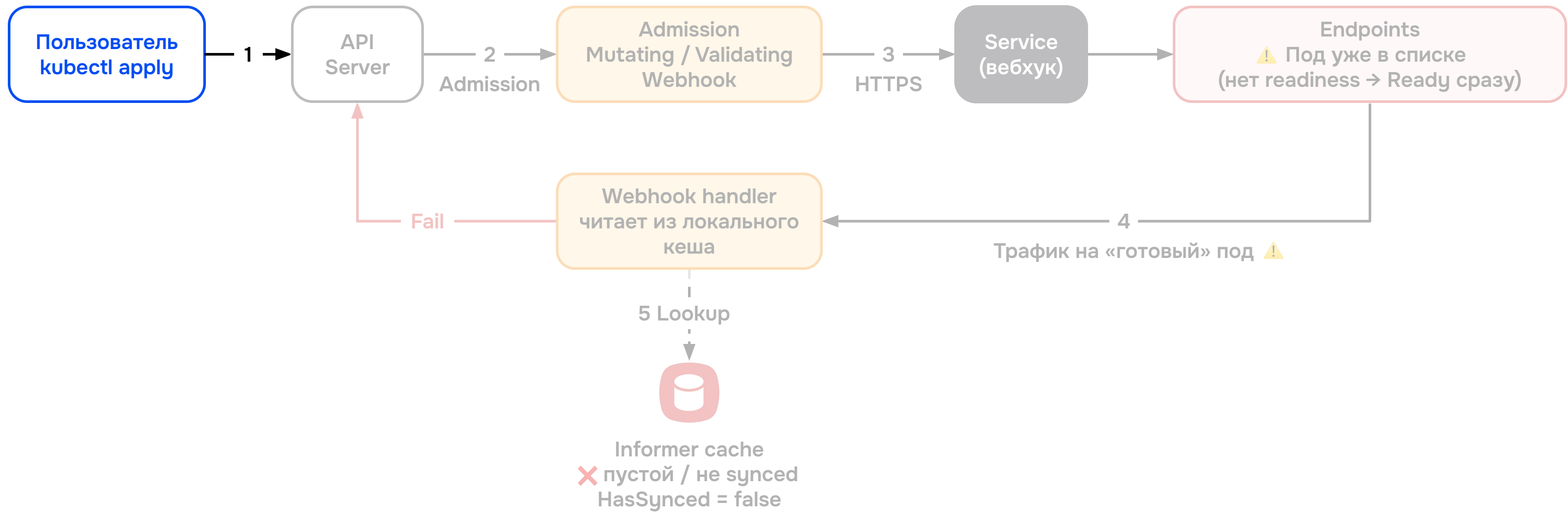
Какие есть пробы и для чего?

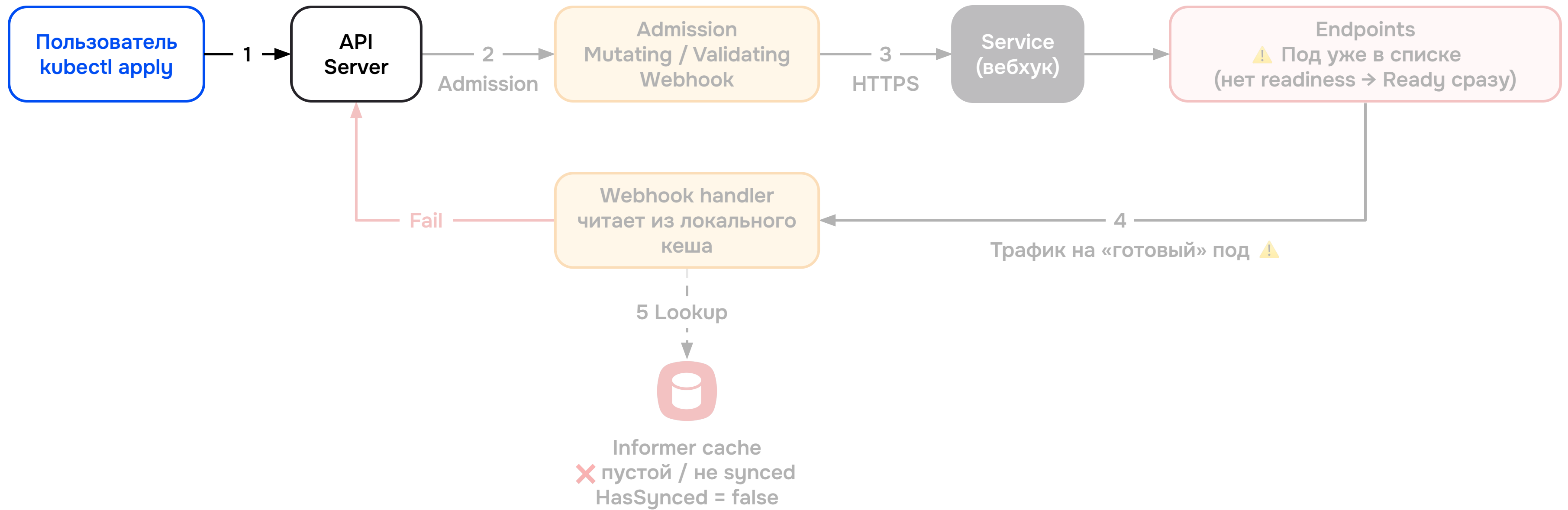
- 01 **Startup** – даёт приложению время на долгий старт: пока probe не прошла, liveness и readiness не запускаются
- 02 **Readiness** – показывает, готов ли под сейчас принимать работу: при fail – под снимается с балансировки (нет в Endpoints)

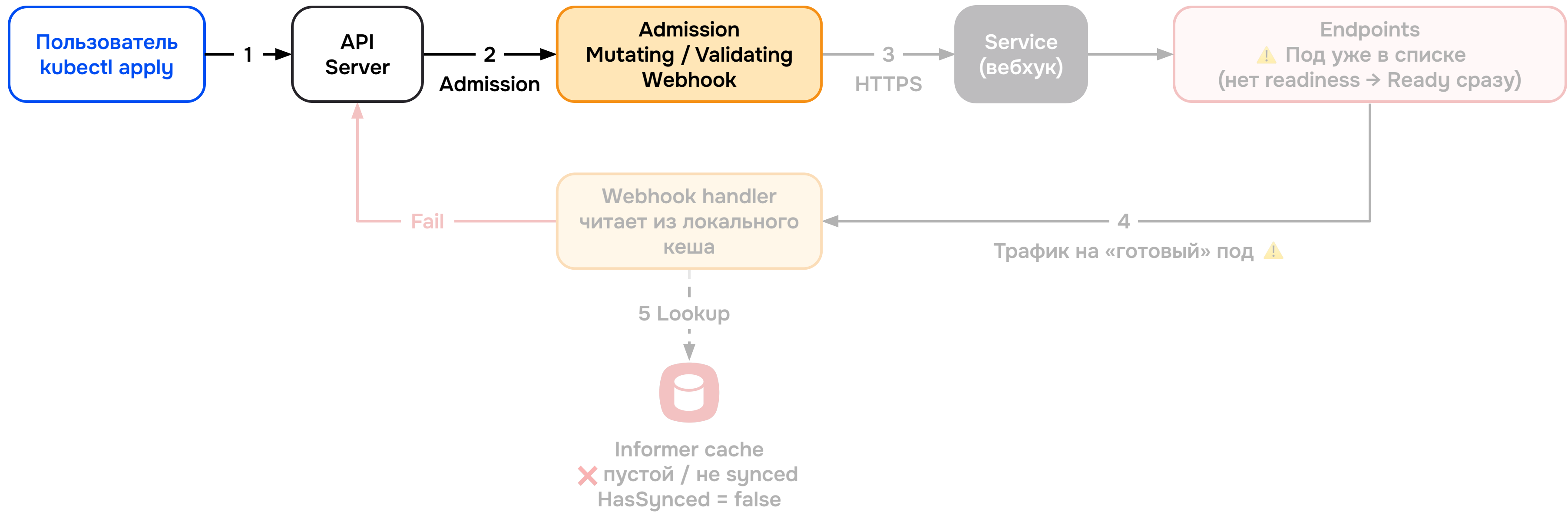
Какие есть пробы и для чего?

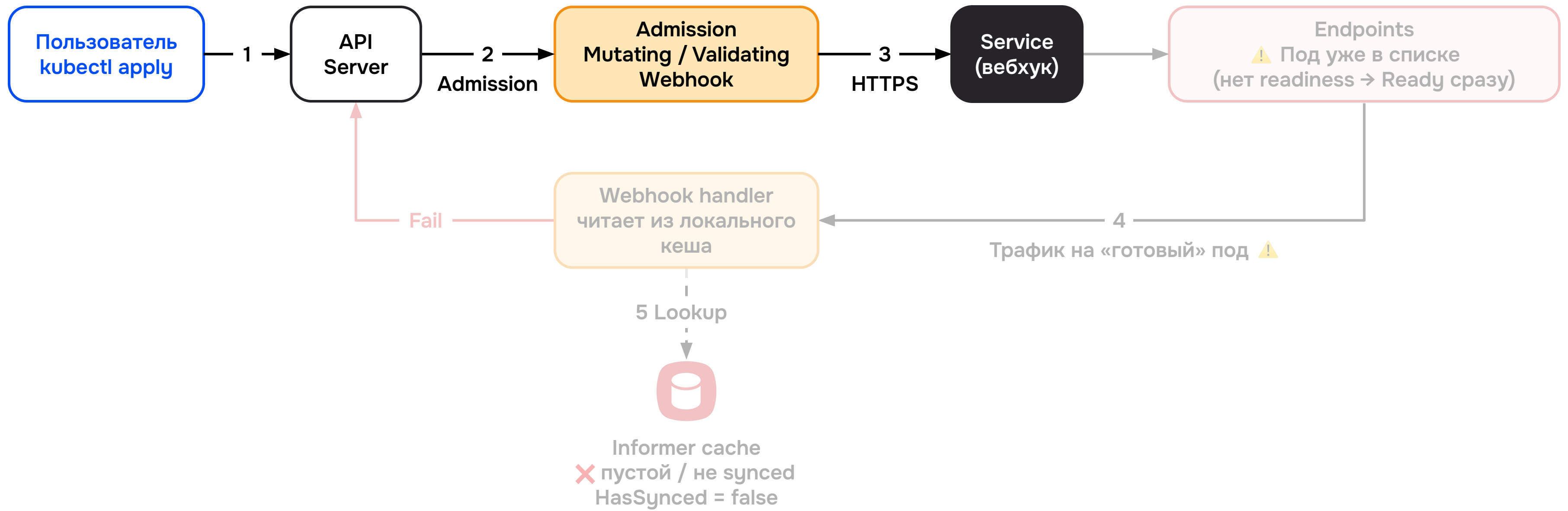
- 01 **Startup** – даёт приложению время на долгий старт: пока probe не прошла, liveness и readiness не запускаются
- 02 **Readiness** – показывает, готов ли под сейчас принимать работу: при fail – под снимается с балансировки (нет в Endpoints)
- 03 **Liveness** – проверяет, что процесс жив: при fail – kubelet перезапускает контейнер

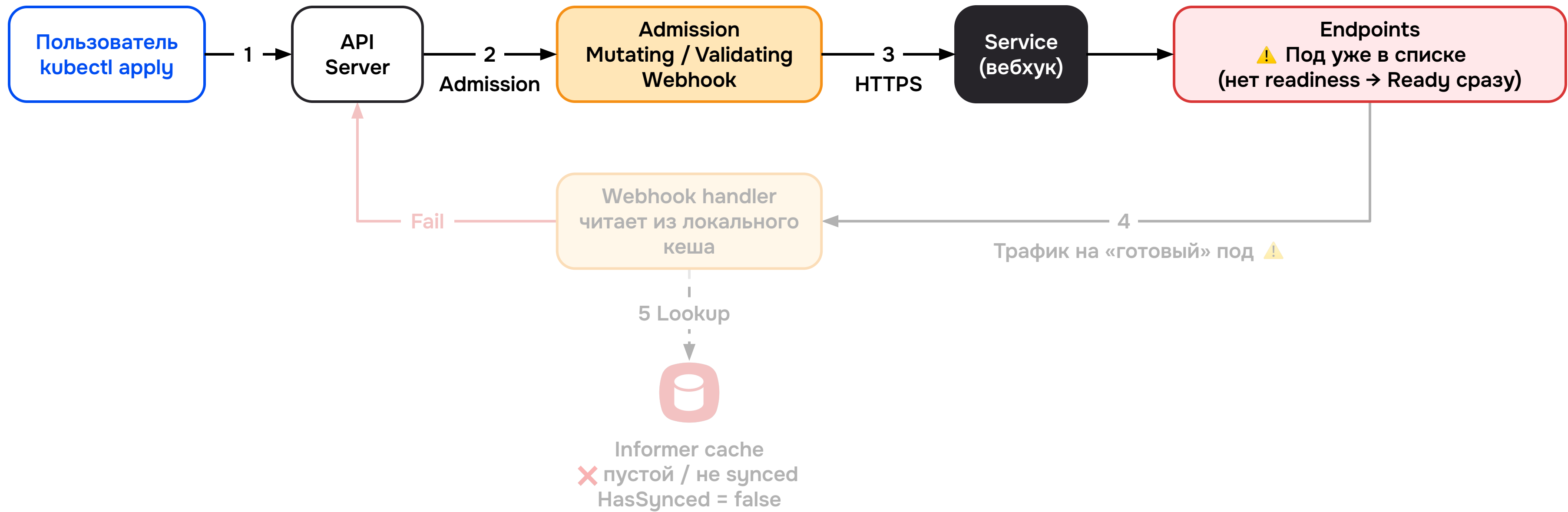
Что происходит, если пробы нет,
а кеш не прогрет?

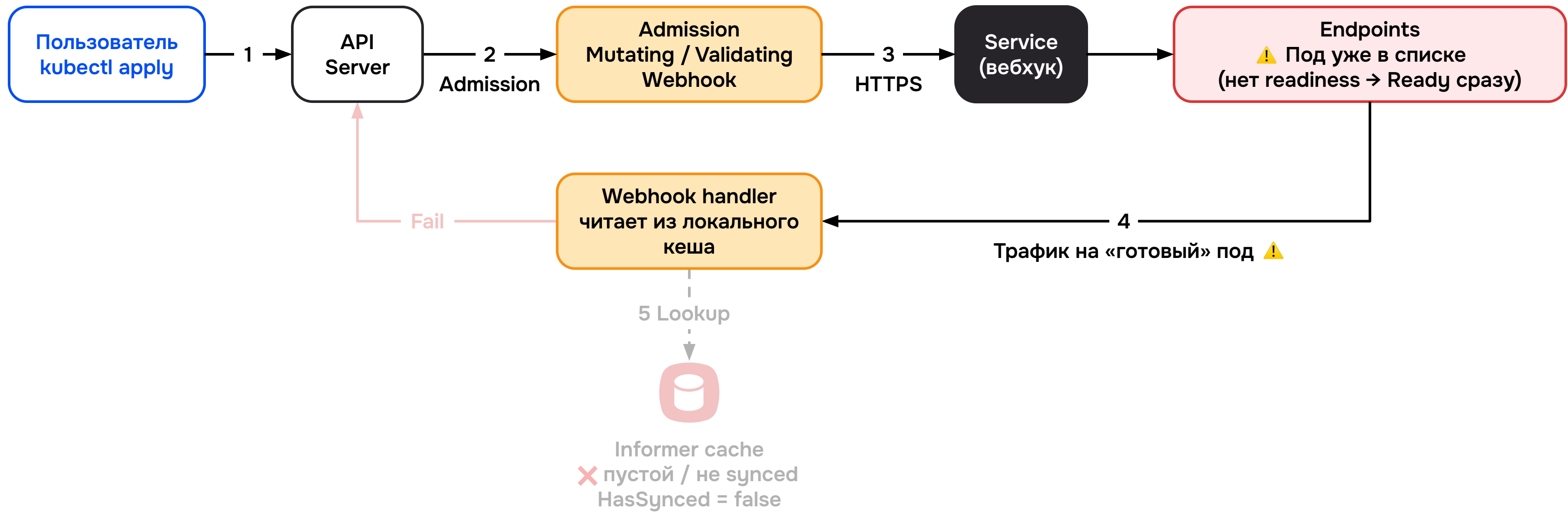


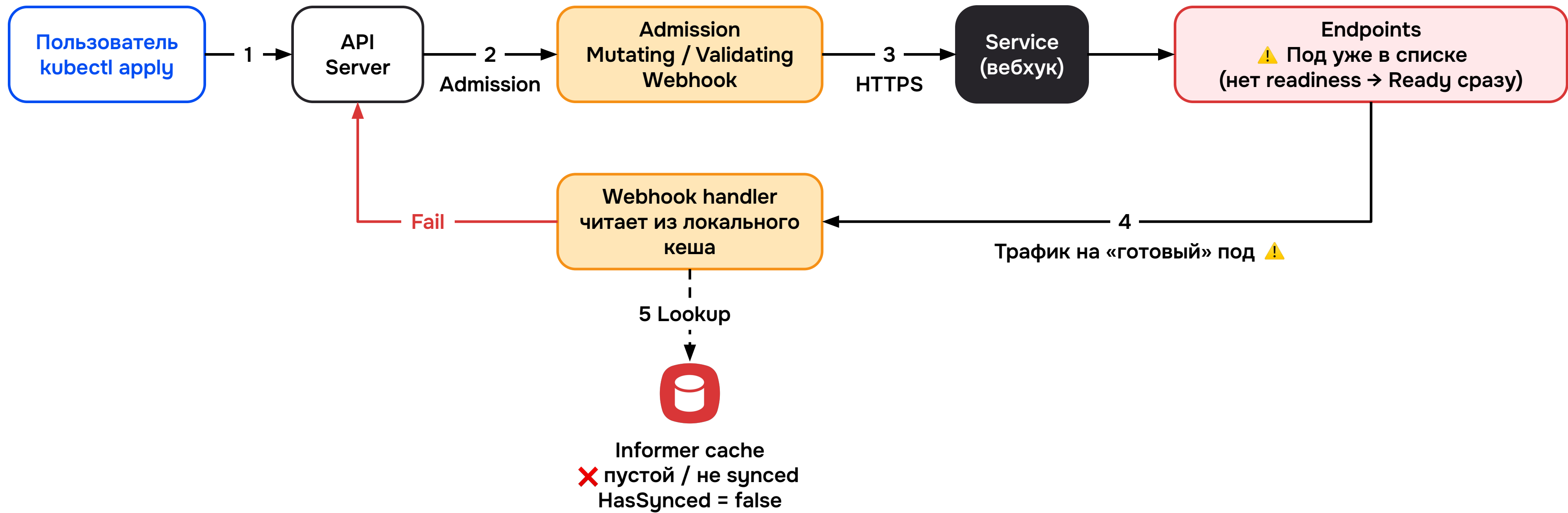






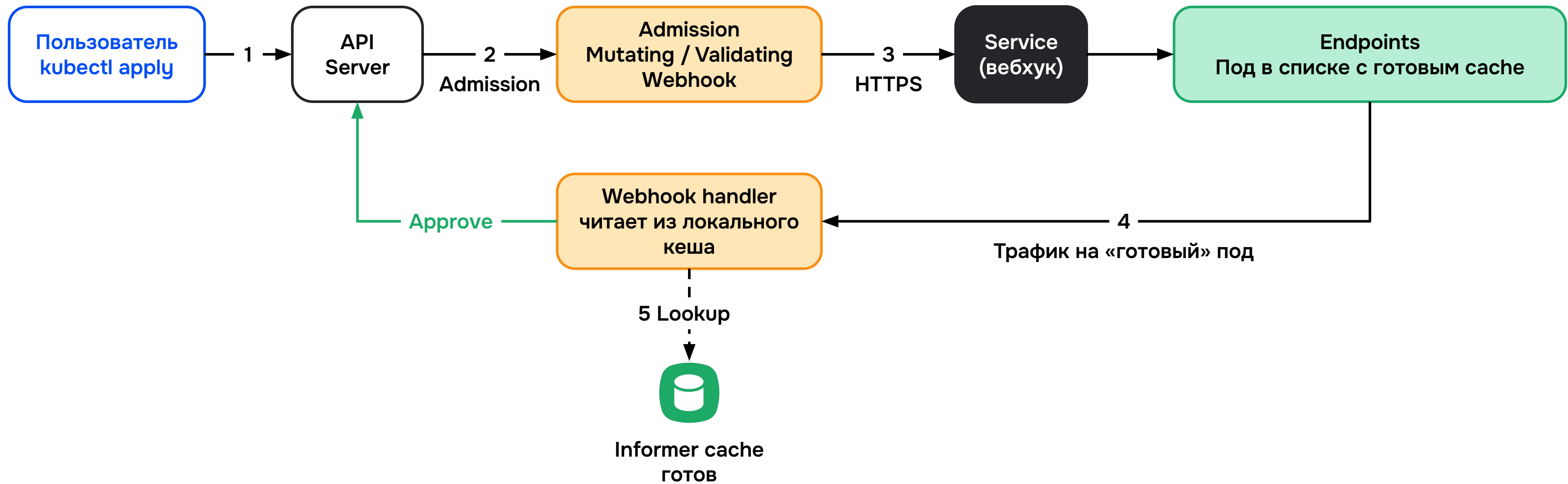






Решение

```
func AddInformerCacheReadyzCheck(mgr ctrl.Manager) error {  
    informerSynced := make(chan struct{})  
    go func() {  
        if mgr.GetCache().WaitForCacheSync(context.Background()) {  
            close(informerSynced)  
        }  
    }()  
  
    return mgr.AddReadyzCheck("informer-cache", func(_ *http.Request) error {  
        select {  
        case <-informerSynced:  
            return nil  
        default:  
            return errors.New("cache not synced yet")  
        }  
    })  
}
```



Айсберг

Lazy Informers

Overload Cache

Probes

Errors maintance

Queue prioritization

Indexers

Rate limiters

Predicates

Expectations pattern

Выводы

Выводы

- 01 Есть библиотеки и SDK, которые ускоряют разработку операторов, но использовать дефолтные настройки библиотек можно не всегда
- 02 Не нужно использовать динамическое создание информеров в рантайме
- 03 Чтобы предотвращать утечки памяти при старте оператора, необходимо настраивать фильтрацию для кеша
- 04 Нужно использовать механизм проб K8s, чтобы посылать трафик в готовый под



Тест



Библиотека

Оцените доклад →

 @deckhouse_news

 github.com/deckhouse

